



CSS Grid

1. CSS Grid hai kya?

CSS Grid ek **2D layout system** hai — matlab tum ek hi time pe **rows aur columns dono** control kar sakte ho. Flexbox 1D hai (ya row ya column, ek time pe ek), lekin Grid dono dimensions ek saath sambhalta hai. Page layouts, dashboards, galleries, card grids — sab ke liye perfect.

Do players hote hain:

- **Grid container** — parent element jisme `display: grid` lagta hai
- **Grid items** — uske direct children, jo grid ke andar place hote hain

2. Basic setup

```
<div class="container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
  <div>6</div>
</div>
```

```
.container {
  display: grid;
  grid-template-columns: 100px 100px 100px; /* 3 columns, har ek 100px */
  grid-template-rows: 80px 80px; /* 2 rows, har e
```

```
k 80px */
}
```

Bas itna karte hi 6 divs 3×2 ka grid bana lenge. Container ke andar wale items automatically left-to-right, top-to-bottom fill ho jaate hain.

3. **fr** unit — Grid ka superpower

fr matlab **fractional unit** — bachi hui available space ka fraction. Ye Grid ka sabse important concept hai.

```
.container {
  display: grid;
  grid-template-columns: 1fr 2fr 1fr;
  /* Total 4 parts: pehla 1/4, doosra 2/4, teesra 1/4 */
}
```

Agar container 800px ka hai, to columns honge 200px, 400px, 200px. Window resize karo, automatically adjust ho jayega. Mix bhi kar sakte ho:

```
grid-template-columns: 200px 1fr 1fr;
/* Pehla fixed 200px, baaki bachi space 2 equal parts mein */
```

4. **repeat()** — Repetition se bachao

Agar 12 equal columns chahiye, to `1fr 1fr 1fr 1fr 1fr 1fr 1fr 1fr 1fr 1fr 1fr 1fr` likhna pagalpan hai. Iske liye `repeat()`:

```
grid-template-columns: repeat(12, 1fr); /* 12 equal columns
*/
grid-template-columns: repeat(3, 200px); /* 3 columns × 200p
x */

/* Mix bhi chalega: */
grid-template-columns: 100px repeat(3, 1fr) 100px;
```

```
/* Pehla 100px, beech mein 3 flexible, last 100px (header/sid  
ebar pattern) */
```

5. **gap** — Cells ke beech spacing

Margin lagana bhul jao. Grid mein **gap** use karo:

```
.container {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  gap: 20px;           /* row aur column dono mein 20px */  
  row-gap: 10px;       /* sirf rows ke beech */  
  column-gap: 30px;    /* sirf columns ke beech */  
  gap: 10px 30px;      /* row-gap column-gap shorthand */  
}
```

6. Grid lines — Items ko exact jagah pe rakhna

Pehle ke diagram mein dekha — har column aur row ke around numbered lines hoti hain. 3 columns wale grid mein **4 vertical lines** hoti hain (1, 2, 3, 4). Inhi lines ko use karke items place karte hain:

```
.item {  
  grid-column-start: 1;  
  grid-column-end: 3;    /* line 1 se line 3 tak = 2 columns  
occupy */  
  grid-row-start: 1;  
  grid-row-end: 2;  
}  
  
/* Shorthand: */  
.item {  
  grid-column: 1 / 3;    /* start / end */
```

```
grid-row: 1 / 2;
}
```

span keyword — Aur bhi clean

```
.item {
  grid-column: 1 / span 2; /* line 1 se start, 2 columns span karo */
  grid-row: span 3;        /* 3 rows occupy karo, jahan se bhi start ho */
}
```

Negative line numbers — Right/bottom se count

```
.item {
  grid-column: 1 / -1; /* poori width — line 1 se last line tak */
}
```

- 1 matlab last line. Full-width row banane ke liye sabse useful trick hai ye.

7. **grid-template-areas** — Sabse intuitive feature

Ye CSS Grid ka **sabse pyara feature** hai. Layout ko literally ASCII art ki tarah likh sakte ho:

```
.container {
  display: grid;
  grid-template-columns: 200px 1fr 150px;
  grid-template-rows: 80px 1fr 60px;
  grid-template-areas:
    "header header header"
    "sidebar main aside"
    "footer footer footer";
  gap: 10px;
```

```

    min-height: 100vh;
}

.header { grid-area: header; }
.sidebar { grid-area: sidebar; }
.main    { grid-area: main; }
.aside   { grid-area: aside; }
.footer  { grid-area: footer; }

```

Dekh kar hi pata chal jata hai ki layout kya hai — header top mein full-width, sidebar/main/aside beech mein, footer neeche full-width. Same naam ko adjacent cells mein repeat karne se wo area span ho jata hai. Empty cell ke liye `.` (dot) use karo.

8. `minmax()` — Flexible with limits

Cells ko ek minimum aur maximum size ke beech rakhna ho:

```

grid-template-columns: minmax(200px, 1fr) 3fr;
/* Pehla column kabhi 200px se chhota nahi hoga, lekin 1fr tak grow karega */

```

Cards ki height same rakhne ke liye:

```

grid-auto-rows: minmax(150px, auto);
/* Har row minimum 150px, content zyada hai to grow ho jayega */

```

9. Alignment — 6 properties, 2 dimensions

Yahaan thoda confuse hota hai logon ko, lekin pattern simple hai. **2 axes hain:**

- `justify-*` → **horizontal** (inline / row axis)
- `align-*` → **vertical** (block / column axis)

Aur **3 levels:**

Container pe individual items align karna:

```
.container {  
  justify-items: center; /* har item apne cell mein horizon  
tally center */  
  align-items: center; /* har item apne cell mein vertica  
lly center */  
  place-items: center; /* dono ek saath (shorthand) */  
}
```

Pure grid ko container ke andar position karna (jab grid container se chhota ho):

```
.container {  
  justify-content: space-between; /* horizontal distribution  
*/  
  align-content: center; /* vertical distribution  
*/  
  place-content: center;  
}
```

Values: `start`, `end`, `center`, `space-between`, `space-around`, `space-evenly`.

Ek specific item ko alag align karna (item pe lagta hai):

```
.special-item {  
  justify-self: end;  
  align-self: start;  
  place-self: start end;  
}
```

12. Real-world example — Complete dashboard layout

Sab kuch milake ek practical example:

```

<div class="dashboard">
  <header>Logo & Nav</header>
  <nav>Sidebar links</nav>
  <main>Main content</main>
  <aside>Notifications</aside>
  <footer>© 2026</footer>
</div>

```

```

.dashboard {
  display: grid;
  grid-template-columns: 220px 1fr 280px;
  grid-template-rows: 64px 1fr 48px;
  grid-template-areas:
    "header header header"
    "nav      main      aside"
    "footer  footer  footer";
  gap: 16px;
  min-height: 100vh;
  padding: 16px;
}

header { grid-area: header; }
nav     { grid-area: nav;   }
main    { grid-area: main;  }
aside   { grid-area: aside; }
footer  { grid-area: footer; }

/* Mobile pe layout change */
@media (max-width: 768px) {
  .dashboard {
    grid-template-columns: 1fr;
    grid-template-rows: auto auto auto auto auto;
    grid-template-areas:
      "header"

```

```
    "nav"  
    "main"  
    "aside"  
    "footer";  
  }  
}
```

Notice kitna readable hai — media query mein bas areas ko reshape karna padta hai, HTML touch nahi karna.

10. Quick responsive card gallery

Ye snippet bohot kaam aayega — koi bhi card listing isse cover ho jati hai:

```
.cards {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));  
  gap: 24px;  
}  
  
.cards > .card {  
  background: white;  
  border-radius: 12px;  
  padding: 20px;  
  box-shadow: 0 2px 8px rgba(0,0,0,0.08);  
}
```

3 lines mein fully responsive grid ban gaya — koi media query nahi, koi calculation nahi.

Quick cheat sheet (yaad rakhne ke liye)

Container pe lagne wali properties:

- `display: grid`
- `grid-template-columns`, `grid-template-rows`

- `grid-template-areas`
- `gap` , `row-gap` , `column-gap`
- `grid-auto-rows` , `grid-auto-columns` , `grid-auto-flow`
- `justify-items` , `align-items` , `place-items`
- `justify-content` , `align-content` , `place-content`

Items pe lagne wali properties:

- `grid-column` , `grid-row` (start / end)
- `grid-area`
- `justify-self` , `align-self` , `place-self`

Practice ke liye tip

Browser DevTools mein **Firefox** ka Grid Inspector kamaal ka hai — lines, areas, gaps sab visible ho jaate hain on hover. Chrome mein bhi Elements panel mein "grid" badge click karke same overlay milta hai. Jab bhi koi grid likho, devtools mein dekho — visual feedback se bahut jaldi seekho ge.

Koi specific concept aur detail mein samjhana hai, ya kisi particular layout ka example chahiye (Pinterest-style masonry, holy grail layout, magazine layout)? Bata do, uspe focused practice karte hain.